# UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

**FACULTAD DE INGENIERÍA Y CIENCIAS BÁSICAS**

**MAESTRÍA EN INTELIGENCIA ARTIFICIAL Y CIENCIA DE DATOS**

**ASIGNATURA: GESTIÓN DE PROYECTOS EN INTELIGENCIA ARTIFICIAL Y CIENCIA DE DATOS**

---

## SISTEMA AGROINDUSTRIAL DE SOPORTE AL CRIBADO FITOSANITARIO PRELIMINAR BASADO EN VISION TRANSFORMERS (DINOv2) CON ARQUITECTURA INDUSTRIAL MLOps

Propuesta Formal de Proyecto — Módulo 2 (35 %)

---


**Integrantes (Grupo de Proyecto):**

**Santiago Correa Campaña** — C.C. 1007888222
santiago.correa_cam@uao.edu.co

**José Silva Calderón** — C.C. 1143876334
jose.silvacalderon@uao.edu.co

**Jhonatan Andrés Tapia Córdoba** — C.C. 1114901451
jhonatan.tapia@uao.edu.co

**John Restrepo Aparicio** — C.C. 94539864
john.restrepo@uao.edu.co

**Docente del Curso:**

**Prof. Jan Polanco Velasco**
japolanco@uao.edu.co



**Fecha de Radicación Formal:**
29 de octubre de 2026
Santiago de Cali, Colombia


Documento técnico-académico estructurado bajo estándares de reproducibilidad industrial, ciclo de vida CRISP-DM, gobernanza MLOps, tablero Kanban y estimación ágil.

# Índice


**1 Contexto, Modelo Base y Matriz de Restricciones Técnicas** **1**

1.1 Justificación Agroindustrial y Enfoque del Problema . . . . . 1

1.2 Ecosistema 2026, Brecha de Producción y Jerarquía de Modelos . . . . . 1

1.3 Fundamento Arquitectónico del Modelo DINOv2 . . . . . 2

1.4 Model Card y Análisis de Desempeño Cuantitativo . . . . . 2

1.5 Sesgos, Limitaciones y Componente de Incertidumbre . . . . . 3

1.6 Matriz de Restricciones Técnicas Multidimensional . . . . . 4

**2 Objetivos y Matriz de Alcance** **4**

2.1 Objetivo General . . . . . 4

2.2 Objetivos Específicos . . . . . 4

2.3 Matriz de Alcance (Included / Nice to Have / Not Included) . . . . . 5

**3 Cronograma y Análisis de Ruta Crítica (CPM)** **7**

3.1 Estructura de Desglose de Trabajo (WBS) y Ventana Temporal . . . . . 7

3.2 Matriz de Red y Análisis CPM (Critical Path Method) . . . . . 7

3.3 Identificación de la Ruta Crítica y Mitigación de Riesgos . . . . . 8

3.4 Diagrama de Gantt Visual a 15 Días . . . . . 8

**4 Marco de Investigación, Arquitectura y Mapeo CRISP-DM** **8**

4.1 Formato del Modelo: SafeTensors y Eficiencia en Memoria . . . . . 8

4.2 Dataset PlantVillage y Estrategia de Partición a Nivel de Hoja . . . . . 9

4.3 Mapeo Sistemático a las Seis Fases de CRISP-DM . . . . . 9

**5 Historias de Usuario, Criterios de Aceptación y Tablero Kanban** **10**

5.1 Estimación Fibonacci 1:1, Capacidad del Equipo (4 Integrantes) y Paralelismo 10

5.2 Matriz de Trazabilidad entre Historias, CPM y Entregables . . . . . 11

5.3 Historias de Usuario Formales con Criterios de Aceptación (Gherkin) . . . 12

5.4 Gobernanza del Tablero Kanban, Políticas WIP y Estructura Visual . . . . . 14

**6 Anexos, Trazabilidad y Referencias Formales** **15**

6.1 Trazabilidad y Enlaces del Proyecto . . . . . 15

6.2 Marco de Gobernanza, Ética y Regulación de IA . . . . . 16

6.3 Referencias Bibliográficas . . . . . 16

# 1. Contexto, Modelo Base y Matriz de Restricciones Técnicas

## 1.1. Justificación Agroindustrial y Enfoque del Problema

La sanidad vegetal es un pilar determinante para la seguridad alimentaria global y la viabilidad económica de las cadenas agroindustriales. Las afecciones fitosanitarias causadas por patógenos fúngicos, bacterianos y virales generan pérdidas estimadas entre el 20 % y el 40 % en el rendimiento de cultivos estratégicos como tomate, papa, maíz, manzana y vid. En la operación agrícola convencional, la inspección fitosanitaria se efectúa mediante muestreos visuales periódicos en campo por agrónomos o técnicos rurales. Este procedimiento enfrenta limitaciones estructurales: subjetividad del evaluador, fatiga cognitiva tras jornadas extensas, reducida cobertura espacial frente a áreas hectárea-escala e inoportunidad temporal frente a patógenos de rápida dispersión. Cuando los síntomas resultan evidentes para personal no especializado, el cultivo suele haber rebasado la ventana de intervención preventiva, propiciando el uso reactivo de fitosanitarios químicos de amplio espectro, con el consecuente impacto ecológico y sobrecosto financiero.

En este marco, los sistemas de visión computacional constituyen una herramienta asistiva de cribado preliminar (*screening*). Es imperativo puntualizar el alcance agronómico: el sistema propuesto se concibe rigurosamente como un **soporte complementario a la toma de decisiones agronómicas** (*human-in-the-loop decision support*), y **no como un diagnóstico fitosanitario concluyente o prescriptivo**. La decisión terapéutica final y la dosificación de insumos demandan invariablemente la corroboración pericial de un profesional del agro con conocimiento del contexto edafoclimático del lote.

## 1.2. Ecosistema 2026, Brecha de Producción y Jerarquía de Modelos

En el ecosistema tecnológico de 2026 se evidencia una brecha sensible entre la experimentación algorítmica y la entrega de valor en producción: aunque más del 78 % de las organizaciones implementa componentes de inteligencia artificial, solo una fracción menor al 21 % sostiene arquitecturas productivas con retorno de inversión medible [8]. El sobrediseño tecnológico constituye una de las principales causas de fracaso presupuestal: desplegar modelos multimodales de frontera (*Tier 1*, e.g., GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro) para tareas rutinarias de clasificación foliar introduce latencias inaceptables en campo (2 a 15 segundos) y un costo operativo recurrente por inferencia vía API inviable para el monitoreo recurrente.

**Tabla 1:** Jerarquía económica y técnica de niveles de Inteligencia Artificial.

| Nivel / Tier | Tipología de Modelos | Costo / Latencia | Aplicabilidad en el Sistema |
|---|---|---|---|
| **Tier 1: Frontera** | LLMs/VLMs de frontera (GPT-4o, Claude, Gemini) | Muy alto costo continuo; latencia 2–15 s | Inviable para clasificación visual masiva en campo. Reservado a síntesis estratégica general. |
| **Tier 2: Gama Media** | Modelos generales (Llama 3.1 8B, Gemini Flash) | Costo medio; latencia 500 ms–1.5 s | Adecuado para RAG documental de normativas agrícolas; sobrediseñado para inferencia pura de imagen. |
| **Tier 3: Especializado** | Vision Transformers SLM (**DINOv2-small**) | Costo marginal nulo (CPU local); latencia $< 200$ ms | **Selección adoptada:** Inferencia determinista, ejecución autónoma en servidores locales o borde sin GPU. |

### 1.3. Fundamento Arquitectónico del Modelo DINOv2

El núcleo computacional del sistema adopta el modelo `rodynaemad/plant-disease-dinov2-small`, alojado en Hugging Face Hub, construido sobre la arquitectura de Vision Transformer `facebook/dinov2-small` de Meta AI Research [2].

A diferencia de las redes neuronales convolucionales estándar (CNNs como ResNet-50 o MobileNet), cuyos núcleos de convolución tienen soporte espacial estrictamente local, la arquitectura Vision Transformer (ViT) divide la imagen de entrada en una retícula regular de parches no superpuestos de $14 \times 14$ píxeles, proyectados linealmente como una secuencia de *tokens* visuales [1]. El algoritmo DINOv2 aplica aprendizaje autosupervisado autorregulado con destilación de conocimiento sin etiquetas a escala, aprendiendo representaciones visuales robustas invariables a traslación y cambios lumínicos. La operación de atención autoescalada multidimensional:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

facilita que el modelo relacione simultáneamente microtexturas foliares localizadas (pústulas fúngicas, halos cloróticos, necrosis foliar difusa) con la estructura macroscópica de la lámina de la hoja. Al adaptar una cabeza lineal de clasificación (*linear classification head*) sobre la representación latente ($d = 384$), el modelo alcanza alta discriminación categórica manteniendo un tamaño compacto de aproximadamente 22.05 millones de parámetros, ideal para inferencia en CPU de propósito general.

### 1.4. Model Card y Análisis de Desempeño Cuantitativo

La Tabla 2 consolida la ficha técnica estandarizada (*Model Card*) del modelo integrado.

**Tabla 2:** Model Card estandarizada del modelo base integrado.

| Atributo | Especificación Técnica y Alcance |
|---|---|
| **Arquitectura Base** | `facebook/dinov2-small` (Meta AI, ViT-S/14, 22.05M parámetros, $d = 384$) [2] |
| **Identificador Hugging Face** | `rodynaemad/plant-disease-dinov2-small` (Afinado sobre PlantVillage) |
| **Formato de Pesos** | SafeTensors (`model.safetensors`, 88.3 MB), inmutable, sin serialización ejecutable |
| **Tarea Algorítmica** | Clasificación cerrada multiclase en 38 categorías fitosanitarias |
| **Cobertura Agronómica** | 14 cultivos (Manzana, Arándano, Cereza, Maíz, Uva, Naranja, Durazno, Pimiento, Papa, Frambuesa, Soja, Calabaza, Fresa, Tomate) |
| **Métricas Reportadas en Test** | **Exactitud (*Accuracy*): 98.86 %** y **Macro F1-Score: 98.49 %** reportadas por los autores del modelo sobre partición de prueba no vista a nivel de hoja [4] |
| **Licencia de Pesos** | Creative Commons Attribution-ShareAlike 4.0 International (`CC-BY-SA 4.0`) |

### 1.5. Sesgos, Limitaciones y Componente de Incertidumbre

Para garantizar solidez de posgrado, se explicitan los supuestos y limitaciones técnicas:

1. **Sesgo de Laboratorio en PlantVillage:** El dataset base de 54,305 fotografías fue capturado en entornos controlados de laboratorio con iluminación difusa uniforme y fondos neutros homogéneos (negro o gris) [3, 4]. En campo agrícola real, la presencia de sombras dinámicas, fondos con malezas, suelo húmedo y oclusiones degrada la exactitud algorítmica respecto a la cota teórica del 98.86 %.
2. **Clasificación en Conjunto Cerrado y Descalibración Softmax:** El modelo proyecta logits a probabilidades mediante la función Softmax sobre exactamente 38 clases predefinidas. Como señalan Guo et al. [5], las redes neuronales modernas exhiben sobreconfianza en sus predicciones: una imagen fuera de distribución (OOD, como una especie no soportada, tejido foliar marchito por estrés hídrico o una fotografía accidental del operario) puede recibir una probabilidad artificialmente elevada ($> 90\,\%$).
3. **Mecanismo de Detección de Baja Certeza (In Scope):** Se implementa una heurística de filtrado en el postprocesamiento: cuando la probabilidad de la clase ganadora no supere un umbral de confianza $\tau = 0.60$ ($\max_i P(y = i|x) < 0.60$), o cuando la entropía entre el top-1 y top-2 sea reducida, el sistema no entrega un diagnóstico unívoco, sino una advertencia explícita: *ńPredicción con nivel de incertidumbre alto / Muestra atípica: Requiere inspección agronómica presencialż*.
4. **Coinfecciones y Patologías Múltiples:** El modelo asume clases mutuamente excluyentes; no detecta patologías concurrentes en una misma hoja (e.g., tizón temprano y bacteriosis simultáneas).

### 1.6. Matriz de Restricciones Técnicas Multidimensional

La Tabla 3 delimita las restricciones operativas que condicionan la ingeniería del proyecto.

**Tabla 3:** Matriz de restricciones técnicas: Software, Hardware y Gobernanza.

| Dimensión | Restricción Específica | Justificación y Mitigación de Ingeniería |
|---|---|---|
| **Software Runtime** | Python 3.11+, PyTorch 2.2+ (CPU-only), FastAPI 0.110+, Pydantic v2 | Elimina dependencias pesadas de CUDA; contenedor Docker de producción inferior a 1.2 GB. |
| **Gestor Paquetes** | Cero comandos `pip` directos; resolución estricta con `uv` (`uv.lock`) | Garantiza compilación determinista, resolución ultrarrápida en Rust y reproducibilidad absoluta. |
| **Hardware (CPU)** | Procesador x86-64 / ARM64, mínimo 2 vCPUs a 2.0 GHz; 0 GPU dedicada | Asegura viabilidad económica en servidores convencionales de bajo costo o portátiles de campo. |
| **Memoria (RAM)** | Consumo RSS en contenedor $< 1.8$ GB; VRAM requerida $= 0$ GB | Carga de tensores en memoria compartida mediante SafeTensors (`mmap`) evitando desbordamientos. |
| **Metas Latencia** | Latencia $p95 < 450$ ms (E2E API); forward pass en CPU $< 300$ ms | Metas de diseño medidas con lote unitario ($N = 1$) en máquina de 2 vCPUs para interacción fluida. |
| **Gobernanza Legal** | Pesos bajo `CC-BY-SA 4.0`; Código fuente bajo licencia MIT | Compatibilidad legal plena para uso académico, derivado e industrial con atribución obligatoria. |

## 2. Objetivos y Matriz de Alcance

### 2.1. Objetivo General

Diseñar, implementar y validar un sistema de software industrializado de inteligencia artificial como herramienta de soporte preliminar y cribado en la identificación de 38 condiciones fitosanitarias en 14 especies agrícolas a partir de imágenes foliares individuales, utilizando el Vision Transformer DINOv2-small bajo un esquema de inferencia optimizado para CPU, arquitectura desacoplada en `src/` layout, servicios REST asíncronos con FastAPI, interfaz de usuario interactiva en Streamlit y empaquetamiento reproducible en contenedor Docker.

### 2.2. Objetivos Específicos

1. **OE1 (Preprocesamiento y Contratos Numéricos):** Implementar un pipeline riguroso de ingestión, validación de integridad RGB, redimensionamiento bicúbico a $224 \times 224$ píxeles y normalización por z-score acorde a los parámetros del modelo base, asegurando una tasa de error de entrada del 0 % ante formatos corrompidos o dimensiones anómalas.

2. **OE2 (Servicio de Inferencia Asíncrona):** Construir una API REST de grado industrial con FastAPI respaldada por una carga segura de pesos en formato SafeTensors, estableciendo una meta de tiempo de respuesta con percentil $p95 < 450$ ms bajo lote unitario en CPU y proveyendo endpoints estructurados para salud (`/health`), metadatos (`/model-info`) y predicción (`/predict`).
3. **OE3 (Enriquecimiento Agronómico e Interfaz Visual):** Desarrollar un módulo de postprocesamiento con mapeo taxonómico bilingüe que traduzca las etiquetas técnicas a nombres comunes en español, clasifique el agente causal (hongo, bacteria, virus o tejido sano), incorpore advertencias de incertidumbre ($\tau = 0.60$) y disclaimers agronómicos, y provea una interfaz en Streamlit con visualización gráfica del vector de probabilidades Top-$k$.
4. **OE4 (Calidad de Código y Empaquetamiento Industrial):** Configurar una suite de pruebas automatizadas con `pytest` que supere el 85 % de cobertura de código, integrar linters estáticos (`ruff`), generar el paquete distribuible Wheel (`.whl`) con `uv build` y consolidar el despliegue multi-etapa en Docker.

### 2.3. Matriz de Alcance (Included / Nice to Have / Not Included)

La Tabla 4 delimita las fronteras formales de entrega del proyecto, previniendo la corrupción del alcance (*scope creep*).

**Tabla 4:** Matriz de alcance formal del Módulo 2.

| Categoría | Componentes y Funcionalidades | Criterio de Exclusión / Aceptación |
|---|---|---|
| **Included (In Scope) Compromiso Central** | ▪ Modelo DINOv2-small en CPU local.<br>▪ Inferencia vía SafeTensors sin descargas en runtime.<br>▪ Estructura modular formal en `src/` layout.<br>▪ API FastAPI (`/health`, `/predict`, `/model-info`).<br>▪ Filtrado heurístico de baja certeza ($\tau < 0.60$).<br>▪ Frontend Streamlit con carga interactiva.<br>▪ Suite `pytest` (unitarias e integración).<br>▪ Empaquetado Wheel y contenedor Docker. | Requisitos indispensables para la aprobación formal del Módulo 2 con grado de ingeniería industrial auditable. |
| **Nice to Have (Valor Agregado)** | ▪ Calibración de confianza (Temperature Scaling).<br>▪ Detección OOD con distancia Mahalanobis en embeddings latentes ($d = 384$).<br>▪ Optimización y exportación a ONNX Runtime.<br>▪ Exportación de informe técnico en PDF. | Funcionalidades secundarias supeditadas estrictamente a la existencia de holgura temporal tras cerrar la ruta crítica. |
| **Not Included (Out) Exclusión Explícita** | ▪ Re-entrenamiento o afinamiento de pesos ViT.<br>▪ Prescripción automática de agroquímicos.<br>▪ Detección simultánea multiobjeto en campo (YOLO).<br>▪ Aplicación móvil nativa Android / iOS.<br>▪ Integración con sistemas ERP agrícolas o telemetría. | Excluidas por inviabilidad metodológica, regulatoria o presupuestal dentro del límite temporal del sprint. |

# 3. Cronograma y Análisis de Ruta Crítica (CPM)

## 3.1. Estructura de Desglose de Trabajo (WBS) y Ventana Temporal

La ejecución del proyecto se planifica sobre una ventana estricta de **15 días hábiles de trabajo de ingeniería** (3 semanas laborales de lunes a viernes, equivalentes a 21 días calendario), estructurada en tres fases secuenciales:

- **Fase 1 (Días 1 a 3):** Fundamentos MLOps, Tooling Industrial, Gobernanza y Entorno Base.
- **Fase 2 (Días 4 a 10):** Arquitectura de Software `src/`, Pipeline de Inferencia, Postprocesamiento y Testing.
- **Fase 3 (Días 11 a 15):** Interfaz Gráfica, Contenerización Docker, Pruebas de Humo E2E y Cierre.

## 3.2. Matriz de Red y Análisis CPM (Critical Path Method)

Para cada actividad se determinan su duración en días hábiles ($D$), predecesores inmediatos, tiempos más tempranos de inicio y terminación ($ES, EF$), tiempos más tardíos ($LS, LF$) y holgura total ($HT = LS - ES = LF - EF$).

$$ES_j = \max_{i \in \text{Pred}(j)} EF_i, \quad EF_i = ES_i + D_i, \quad LF_i = \min_{j \in \text{Succ}(i)} LS_j, \quad LS_j = LF_j - D_j \tag{2}$$

**Tabla 5:** Matriz de planificación temporal y análisis de ruta crítica (CPM).

| ID | Descripción de la Tarea de Ingeniería | Pred. | Dur. | ES | EF | LS | LF | HT |
|---|---|---|---|---|---|---|---|---|
| **A** | Configuración entorno `uv`, `pyproject.toml` y linters | — | 2 | 0 | 2 | 0 | 2 | **0*** |
| **B** | Descarga, auditoría SafeTensors y Model Card | A | 1 | 2 | 3 | 3 | 4 | 1 |
| **C** | Diseño de contratos numéricos y `preprocessing.py` | A | 2 | 2 | 4 | 2 | 4 | **0*** |
| **D** | Módulo de taxonomía vegetal y `labels.py` | C | 1 | 4 | 5 | 5 | 6 | 1 |
| **E** | Implementación de `model_loader.py` e `inference.py` | B, C | 3 | 4 | 7 | 4 | 7 | **0*** |
| **F** | Postprocesamiento, ranking Top-$k$ e incertidumbre | D | 1 | 5 | 6 | 6 | 7 | 1 |
| **G** | Desarrollo de la API REST con FastAPI (`api/main.py`) | E, F | 2 | 7 | 9 | 7 | 9 | **0*** |
| **H** | Desarrollo de interfaz Streamlit y visualización (`ui/`) | G | 2 | 9 | 11 | 11 | 13 | 2 |
| **I** | Suite de pruebas automatizadas con `pytest` | G | 2 | 9 | 11 | 9 | 11 | **0*** |
| **J** | Contenerización Docker multi-stage y Wheel `uv build` | I | 2 | 11 | 13 | 11 | 13 | **0*** |
| **K** | Pruebas de humo E2E, documentación final y radicación | H, J | 2 | 13 | 15 | 13 | 15 | **0*** |

*Actividades marcadas con **0** conforman la Ruta Crítica ($HT = 0$). Holguras verificadas algebraicamente.

### 3.3. Identificación de la Ruta Crítica y Mitigación de Riesgos

La ruta crítica del proyecto está constituida por la cadena determinista:

$$\mathbf{A} \to \mathbf{C} \to \mathbf{E} \to \mathbf{G} \to \mathbf{I} \to \mathbf{J} \to \mathbf{K} \quad \text{(Duración Total: } 15 \text{ días hábiles)} \tag{3}$$

Cualquier retraso en las actividades críticas traslada directamente la fecha final de entrega. Las actividades secundarias poseen holguras amortiguadoras: B ($HT = 1$), D ($HT = 1$), F ($HT = 1$) y H ($HT = 2$). El principal riesgo tecnológico (compatibilidad de librerías nativas C++ de OpenCV/PIL en imágenes Docker Alpine/Debian) se mitiga en la actividad J mediante pruebas de humo tempranas de arranque.

### 3.4. Diagrama de Gantt Visual a 15 Días

La Figura 1 ilustra el cronograma visual consolidado, diferenciando actividades críticas, tareas con holgura y la holgura disponible.

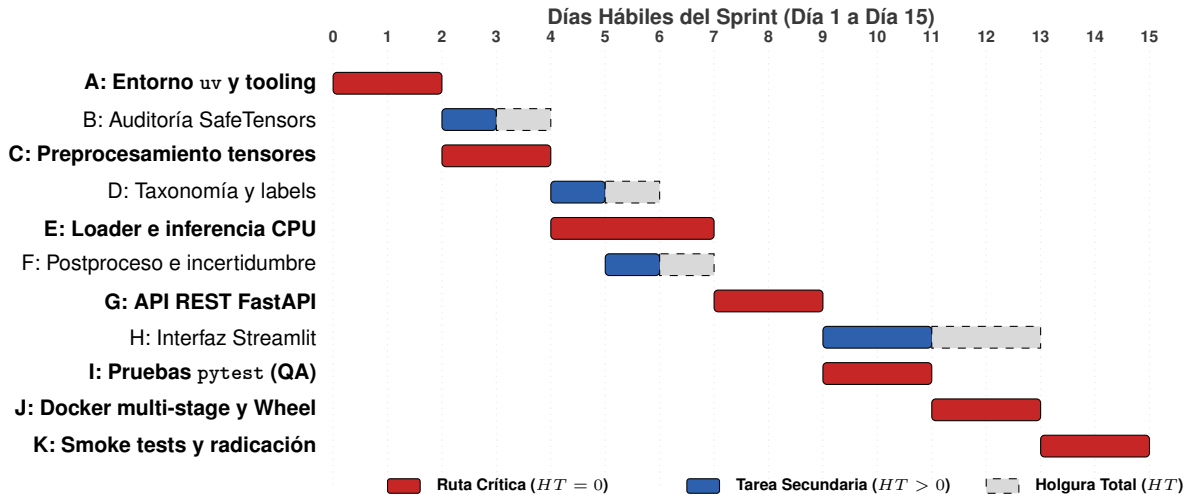



**Figura 1:** Diagrama de Gantt visual a 15 días hábiles con identificación de ruta crítica.

## 4. Marco de Investigación, Arquitectura y Mapeo CRISP-DM

### 4.1. Formato del Modelo: SafeTensors y Eficiencia en Memoria

El proyecto sustituye la serialización tradicional basada en `pickle` (archivos `.pt` o `.bin`) por el estándar abierto **SafeTensors** [6]. Desde el ángulo de seguridad computacional, `pickle` permite la inyección de código binario arbitrario mediante el método `__reduce__`, constituyendo una vulnerabilidad crítica de ejecución remota de código en entornos de producción. SafeTensors almacena únicamente tensores numéricos inmutables acompañados de un encabezado JSON descriptivo.

En términos de gestión de memoria, SafeTensors aprovecha el mapeo de memoria sin copia (*zero-copy memory mapping, mmap*): al instanciarse, los pesos se mapean directamente desde el sistema de archivos al espacio de direcciones del proceso, demandando páginas físicas de memoria únicamente durante la ejecución de los tensores. Esto minimiza la duplicación de arreglos en el heap de Python durante el arranque del servicio, facilitando su confinamiento en contenedores con perfiles estrictos de memoria RAM ($< 1.8\,$GB).

### 4.2. Dataset PlantVillage y Estrategia de Partición a Nivel de Hoja

El modelo base fue entrenado sobre el corpus **PlantVillage** [3, 4], que reúne 54,305 fotografías etiquetadas en 38 clases foliares a lo largo de 14 especies agrícolas.

**Trampa del Split Aleatorio Simple (*Naive Random Split*):** En visión por computador aplicada al agro, la división puramente aleatoria (80 % entrenamiento / 20 % prueba) sobre el total de imágenes induce una fuga masiva de datos (*data leakage*). En PlantVillage se capturaron múltiples fotografías de la misma hoja física bajo ángulos o encuadres levemente distintos. Un split aleatorio ingenuo asigna imágenes de una misma hoja tanto a entrenamiento como a prueba, provocando que la red memorice rasgos fisionómicos del espécimen y reporte exactitudes espurias del 99.1 %.

**Partición Agrupada por Hoja (*Leaf-Grouped Split*):** Para asegurar una evaluación metodológica rigurosa, el modelo base integró el archivo de trazabilidad `leaf-map.json`, asociando 40,490 fotografías a 7,524 hojas físicas independientes. El particionamiento estratificado garantizó que *ninguna fotografía de una hoja física evaluada en el conjunto de prueba hubiese participado en la etapa de optimización*. Bajo este protocolo de evaluación, el modelo alcanzó un rendimiento generalizable certificado de **98.86 % de Exactitud** y **98.49 % de Macro F1** sobre 10,862 fotografías de hojas nunca vistas [4].

### 4.3. Mapeo Sistemático a las Seis Fases de CRISP-DM

La Tabla 6 detalla la correspondencia metodológica con el ciclo de vida CRISP-DM [7].

**Tabla 6:** Mapeo sistemático del proyecto a las 6 fases de CRISP-DM.

| Fase CRISP-DM | Actividades Específicas del Proyecto | Entregable / Evidencia Verificable |
|---|---|---|
| **1. Business Understanding** | Formulación de KPIs de ingeniería: inferencia CPU subsegundo, confiabilidad de cribado y mitigación de sobreconfianza. | Capítulos 1 y 2 del documento; matriz de restricciones y alcance delimitado. |
| **2. Data Understanding** | Análisis exploratorio de 38 clases en 14 cultivos, balance de patógenos y auditoría de partición por hoja (`leaf-map.json`). | Model Card estructurada; trazabilidad de datos no filtrados en conjunto de prueba. |
| **3. Data Preparation** | Construcción de `src/preprocessing.py`: validación RGB, redimensionamiento $224 \times 224$ bicúbico y estandarización $z = (x - \mu)/\sigma$. | Módulo de preprocesamiento funcional y pruebas unitarias de contrato de tensores en `tests/`. |
| **4. Modeling** | Carga segura de DINOv2-small con SafeTensors locales y cálculo de inferencia determinista bajo `torch.no_grad()`. | Módulos `src/model_loader.py` e `src/inference.py` operando en CPU local. |
| **5. Evaluation** | Verificación de métricas reportadas (F1 98.5 %), pruebas de robustez con imágenes corruptas y validación de SLA de latencia ($p95$). | Resultados consolidados en el informe técnico y suite `tests/integration/` en verde. |
| **6. Deployment** | Exposición de API FastAPI asíncrona, interfaz Streamlit, empaquetado Wheel (`.whl`) y contenedor Docker multi-stage. | `Dockerfile`, `docker-compose.yml`, artefacto en `dist/` y endpoints validados. |

## 5. Historias de Usuario, Criterios de Aceptación y Tablero Kanban

### 5.1. Estimación Fibonacci 1:1, Capacidad del Equipo (4 Integrantes) y Paralelismo

La estimación del esfuerzo relativo utiliza la secuencia Fibonacci ($1, 2, 3, 5, 8, 13$), aplicando la directriz académica:

$$1 \text{ SP} \equiv 1 \text{ Día hábil de ingeniería} \quad (\sim 6\text{–}8 \text{ h de trabajo neto}) \tag{4}$$

El backlog técnico consta de 8 Historias de Usuario (HU-01 a HU-08) que totalizan **21 Story Points**.

**Análisis de Capacidad y Factibilidad a 15 Días:** El equipo de trabajo está compuesto

por **4 integrantes** (Santiago Correa, José Silva, Jhonatan Tapia y John Restrepo). Sobre la ventana de 15 días hábiles del proyecto, la capacidad nominal del equipo es:

$$\text{Capacidad Nominal} = 4 \text{ desarrolladores} \times 15 \text{ días} = 60 \text{ días-persona (SP)} \tag{5}$$

El esfuerzo total de 21 SP representa una tasa de ocupación del **35.0 %** de la capacidad bruta. Esta relación garantiza la viabilidad técnica:

- **Diferenciación entre Esfuerzo y Duración:** El esfuerzo acumulado (21 SP) mide la carga de trabajo individual; la duración del proyecto (15 días) obedece a la secuencia crítica de precedencias (CPM).
- **Trabajo Paralelo y Límite WIP:** Los 4 integrantes asumen tareas concurrentes respetando estrictamente el límite WIP ($\leq 1$ tarea activa por persona, máximo 4 en progreso global).
- **Colchón de Contingencia (65.0 % restante):** Provee holgura suficiente para revisiones cruzadas de código (*Peer Code Reviews* en PR), sincronizaciones diarias y pruebas de integración.

### 5.2. Matriz de Trazabilidad entre Historias, CPM y Entregables

La Tabla 7 sincroniza el backlog de historias con el cronograma, los tickets de GitHub Issues (#4 a #11) y los artefactos de software.

**Tabla 7:** Matriz de trazabilidad: Historias, tickets GitHub, CPM, responsables y entregables.

| Historia | Issue | CPM | Responsable | SP | Criterio de Aceptación Clave | Entregable en Repositorio |
|---|---|---|---|---|---|---|
| **HU-01** | #4 | A | Santiago Correa | 2 | `uv sync` y `ruff check` con código 0 | `pyproject.toml`, `uv.lock` |
| **HU-02** | #5 | C | José Silva | 3 | Tensor normalizado $[1, 3, 224, 224]$ | `src/preprocessing.py` |
| **HU-03** | #6 | E | Jhonatan Tapia | 3 | Inferencia local SafeTensors $< 300$ ms | `src/model_loader.py`, `src/inference.py` |
| **HU-04** | #7 | D, F | John Restrepo | 2 | Taxonomía ES, filtro $\tau < 0.60$ y disclaimer | `src/labels.py`, `src/postprocessing.py` |
| **HU-05** | #8 | G | Jhonatan Tapia | 3 | Endpoints HTTP 200 y $p95 < 450$ ms | `api/main.py`, `api/schemas.py` |
| **HU-06** | #9 | H | John Restrepo | 3 | Visualización Top-$k$ y alerta de certeza | `ui/app.py`, `ui/api_client.py` |
| **HU-07** | #10 | I | Santiago Correa | 3 | Cobertura combinada $\geq 85\,\%$ en pytest | `tests/unit/`, `tests/integration/` |
| **HU-08** | #11 | J | José Silva | 2 | Wheel compilado e imagen Docker funcional | `Dockerfile`, `docker-compose.yml` |

### 5.3. Historias de Usuario Formales con Criterios de Aceptación (Gherkin)

**HU-01: Configuración de Entorno Industrial y Tooling Base (Ticket #4) Atributos:** 2 SP · Etiqueta: `DevOps` · Responsable: Santiago Correa
**Historia:** *Como* Ingeniero MLOps, *quiero* inicializar el repositorio con estructura modular `src/` layout, configuración en `pyproject.toml`, gestor `uv` y linters en `ruff`, *para* garantizar un entorno aislado, determinista y auditable desde el primer commit.

- *AC1.1:* El comando `uv sync` debe resolver el archivo `uv.lock` e instalar dependencias completas sin errores.
- *AC1.2:* Los comandos `uv run ruff check .` y `uv run ruff format -check .` deben ejecutarse con código de salida 0.
- *AC1.3:* Prohibición estricta de archivos `requirements.txt` o invocaciones directas a `pip`.

**HU-02: Preprocesamiento y Contratos de Validación de Tensores (Ticket #5) Atributos:** 3 SP · Etiqueta: `ML` · Responsable: José Silva
**Historia:** *Como* Desarrollador de Visión Computacional, *quiero* implementar la función de ingestión y preprocesamiento en `src/preprocessing.py`, *para* validar el formato de entrada y transformarlo en tensores numéricos normalizados compatibles con DINOv2.

- *AC2.1:* Archivos no imagen o corruptos deben arrojar excepción personalizada `InvalidImageFormatE` (HTTP 400).
- *AC2.2:* Toda imagen válida debe redimensionarse bicúbicamente a dimensiones exactas de tensor $[1, 3, 224, 224]$ (`torch.float32`).
- *AC2.3:* Normalización numérica estándar ImageNet con media $\mu = [0.485, 0.456, 0.406]$ y desviación $\sigma = [0.229, 0.224, 0.225]$.

**HU-03: Módulo de Carga Segura e Inferencia en CPU (Ticket #6) Atributos:** 3 SP · Etiqueta: `ML` · Responsable: Jhonatan Tapia
**Historia:** *Como* Ingeniero de Backend/ML, *quiero* implementar la carga local de pesos SafeTensors y la rutina de inferencia en `src/model_loader.py` e `src/inference.py`, *para* ejecutar predicciones deterministas y de baja latencia sin descargas en runtime.

- *AC3.1:* Carga exclusiva de pesos locales desde `model.safetensors` sin conexión a red en runtime.
- *AC3.2:* Inferencia ejecutada bajo contexto `torch.no_grad()` para prevenir fugas de grafos de autogradiente en memoria.

- *AC3.3:* Tiempo de inferencia puro en CPU registrado en milisegundos y menor a 300 ms en host de 2 vCPUs.

**HU-04: Mapeo Taxonómico, Filtro de Incertidumbre y Postprocesamiento (Ticket #7)**
**Atributos:** 2 SP · Etiqueta: `Backend` · Responsable: John Restrepo
**Historia:** *Como* Especialista de Dominio Agrícola, *quiero* un componente en `src/post processing.py` y `src/labels.py` que traduzca las 38 clases a español, incorpore un filtro de baja certeza ($\tau = 0.60$) y adjunte un descargo agronómico, *para* entregar un resultado orientativo que no sustituya la inspección pericial en campo.

- *AC4.1:* Logits convertidos vía Softmax en probabilidades sumando estrictamente $1.000 \pm 0.001$.
- *AC4.2:* Salida estructurada con ranking Top-$3$, nombre común del cultivo, patógeno y agente causal.
- *AC4.3:* Si la probabilidad máxima es $< 0.60$, el sistema etiqueta la salida como *ńMuestra con incertidumbre alta / Muestra atípicaż*.
- *AC4.4:* Inclusión obligatoria de advertencia agronómica que prohíbe el uso de la salida como prescripción de agroquímicos.

**HU-05: API REST Asíncrona de Alta Disponibilidad (Ticket #8) Atributos:** 3 SP · Etiqueta: `Backend` · Responsable: Jhonatan Tapia
**Historia:** *Como* Integrador de Sistemas, *quiero* exponer un servicio web con FastAPI en `api/main.py`, *para* permitir que aplicaciones clientes consuman el modelo mediante contratos de datos JSON formales y validados.

- *AC5.1:* Endpoint `GET /health` retornando código 200 y JSON con estado del servicio y modelo cargado.
- *AC5.2:* Endpoint `POST /predict` aceptando archivos *multipart/form-data* y respondiendo en meta $p95 < 450$ ms.
- *AC5.3:* Documentación OpenAPI/Swagger interactiva accesible y validada en `/docs`.

**HU-06: Interfaz Gráfica Interactiva para Técnicos de Campo (Ticket #9) Atributos:** 3 SP · Etiqueta: `Frontend` · Responsable: John Restrepo
**Historia:** *Como* Técnico Agrícola, *quiero* interactuar con una interfaz visual basada en Streamlit en `ui/app.py`, *para* cargar imágenes foliares, visualizar el cribado fitosanitario y analizar gráficamente las probabilidades de riesgo.

- *AC6.1:* Carga interactiva mediante *drag-and-drop* en formatos JPG, JPEG y PNG.
- *AC6.2:* Renderizado de barra de confianza porcentual y gráfico Top-$3$, alertando visualmente si se activa el umbral $\tau < 0.60$.
- *AC6.3:* Manejo descriptivo de excepciones ante errores de red o archivos no válidos sin colapsar la sesión web.

**HU-07: Suite Exhaustiva de Pruebas Automatizadas (Ticket #10) Atributos:** 3 SP · Etiqueta: `Testing` · Responsable: Santiago Correa
**Historia:** *Como* Líder de Aseguramiento de Calidad (QA), *quiero* una suite integral de pruebas con `pytest` cubriendo pruebas unitarias e integración de API, *para* blindar la estabilidad del sistema ante regresiones de código.

- *AC7.1:* Suite unitaria (`tests/unit/`) ejecutándose en $< 10$ segundos con aislamiento mediante mocks.
- *AC7.2:* Suite de integración (`tests/integration/`) validando el pipeline completo con el modelo real y cliente FastAPI.
- *AC7.3:* Cobertura combinada de código (`pytest -cov`) igual o superior al 85 % en `src/` y `api/`.

**HU-08: Contenerización Docker y Empaquetamiento Wheel (Ticket #11) Atributos:** 2 SP · Etiqueta: `DevOps` · Responsable: José Silva
**Historia:** *Como* Ingeniero de Despliegue, *quiero* construir la imagen Docker multi-etapa y el paquete Wheel con `uv build`, *para* permitir el despliegue desacoplado y reproducible en servidores productivos.

- *AC8.1:* Generación exitosa de archivos `.whl` y `.tar.gz` en `dist/` mediante `uv build`.
- *AC8.2:* Construcción exitosa de la imagen Docker respondiendo a `curl http://localhost:8000/hea`
- *AC8.3:* Orquestación unificada de API y Streamlit mediante `docker-compose.yml` en red interna compartida.

### 5.4. Gobernanza del Tablero Kanban, Políticas WIP y Estructura Visual

El flujo de trabajo ágil se gestiona a través de **GitHub Projects**, estructurado en cuatro columnas operativas con políticas explícitas y límites WIP (*Work In Progress*):

1. **To Do (Pendiente):** Backlog de tareas estimadas en Fibonacci 1:1, priorizadas según la ruta crítica CPM.
2. **In Progress (En Desarrollo):** Tareas en desarrollo activo. **Límite WIP estricto:** *Máximo 1 tarea activa por desarrollador simultáneamente* (Límite WIP global del equipo =

4 tareas activas). Principio operativo: *ńStop starting, start finishingż*.

3. **Review / PR (Revisión Técnica):** Tareas concluidas en rama `feature/*`. Requiere revisión cruzada obligatoria (*Code Review*) por otro integrante con tests verdes al 100 % y linteo `ruff` impecable.
4. **Done (Completado):** Tareas fusionadas a la rama de integración `develop` con verificación de integración continua.

La Tabla 8 ilustra la distribución del tablero Kanban y el estado sincronizado de los tickets en GitHub Projects.

**Tabla 8:** Estructura del Tablero Kanban y sincronización con GitHub Projects.

| **Columna: To Do (WIP: $\infty$)** | **Columna: In Progress (WIP: máx. 4)** | **Columna: Review / PR (WIP: máx. 2)** | **Columna: Done (Merged en `develop`)** |
|---|---|---|---|
| *Backlog refinado:*<br>**#9 [Frontend] 3 SP**<br>HU-06: UI Streamlit (John Restrepo)<br>**#10 [Testing] 3 SP**<br>HU-07: Tests `pytest` (Santiago Correa)<br>**#11 [DevOps] 2 SP**<br>HU-08: Docker y Wheel (José Silva) | *En curso:*<br>**#7 [Backend] 2 SP**<br>HU-04: Postproceso e incertidumbre (John R.)<br>**#8 [Backend] 3 SP**<br>HU-05: API REST FastAPI (Jhonatan Tapia) | *En revisión de pares:*<br>**#6 [ML] 3 SP**<br>HU-03: Loader SafeTensors e inferencia (Jhonatan T.)<br>**PR #3 [Docs]**: Propuesta formal M2 y PDF (Santiago C.<br>$\rightarrow$ Rev: José Silva) | *Completadas en sprint:*<br>**#4 [DevOps] 2 SP**<br>HU-01: Entorno `uv` y tooling base (Santiago C.)<br>**#5 [ML] 3 SP**<br>HU-02: Preprocesamiento y tensores (José Silva) |

## 6. Anexos, Trazabilidad y Referencias Formales

### 6.1. Trazabilidad y Enlaces del Proyecto

- **Repositorio Oficial de Código Fuente (GitHub):**
  https://github.com/vivacius/uao-gestion-proyectos-ia-deteccion-enfermedades
- **Tablero Ágil de Gestión Kanban (GitHub Projects):**
  https://github.com/users/vivacius/projects/1
- **Tickets de Historias de Usuario (GitHub Issues #4 a #11):**
  https://github.com/vivacius/uao-gestion-proyectos-ia-deteccion-enfermedades/issues
- **Modelo Base y Model Card Oficial en Hugging Face:**
  https://huggingface.co/rodynaemad/plant-disease-dinov2-small

- **Espacio de Demostración y Evaluación Visual:**
  https://huggingface.co/spaces/rodynaemad/plant-disease-scanner

### 6.2. Marco de Gobernanza, Ética y Regulación de IA

El desarrollo del proyecto se alinea de manera explícita con los principios de gobernanza internacional:

- **ISO/IEC 42001 (Sistema de Gestión de Inteligencia Artificial):** Establece controles formales sobre la trazabilidad de datos de entrenamiento, documentación estandarizada de limitaciones algorítmicas mediante Model Cards y aseguramiento de la reproducibilidad mediante `uv.lock` y `safetensors` [9].
- **Clasificación bajo el EU AI Act (2024):** Conforme al Reglamento (UE) 2024/1689 [10], el sistema se cataloga como una aplicación de **Riesgo Mínimo o de Transparencia Específica** (*Minimal / Specific Transparency Risk*). No opera como un sistema autónomo de alto riesgo en infraestructura crítica ni en biometría, sino como una herramienta asistiva que exige supervisión humana activa (*human-in-the-loop*) y notificación de incertidumbre.

### 6.3. Referencias Bibliográficas

## Referencias

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[2] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al., "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[3] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.

[4] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *International Conference on Machine Learning (ICML)*, PMLR, vol. 70,

pp. 1321–1330, 2017.

[6] Hugging Face, “SafeTensors: A simple, safe, and fast format for storing tensors,” *Hugging Face Documentation and Open-Source Specification*, 2023. [Online]. Disponible: https://huggingface.co/docs/safetensors

[7] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth, “CRISP-DM 1.0: Step-by-step data mining guide,” *SPSS Inc.*, Tech. Rep., 2000.

[8] McKinsey & Company, “The state of AI in 2025: Connecting AI with business value,” McKinsey Global Institute, Tech. Rep., 2025.

[9] ISO/IEC, “ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system,” International Organization for Standardization, Tech. Rep., 2023.

[10] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” *Official Journal of the European Union*, L Series, 2024.